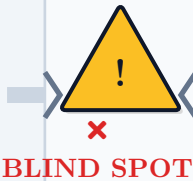


The Mathematical Failure of x -Based Resolutions

Why purely x -descriptors fail completely inside fluid CSS Grid layouts

⚡ The Core Disconnect: Browser Preloader vs. Layout Engine

```
index.html<div class="css-grid-stretchy-container"></div>
```



Mathematical Decision Rule (x -descriptors):

$$\text{Selected File} = f(\text{DPR})$$

Completely independent of CSS layout width!

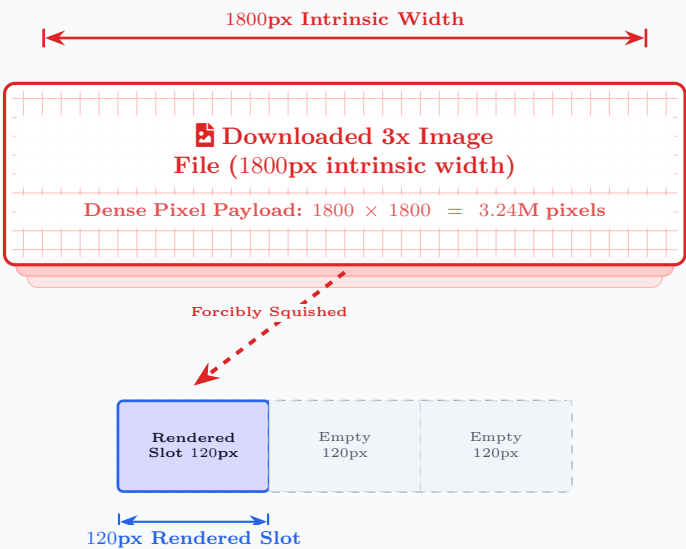
Scenario A: High DPR, Squished Grid Column The Enormous Payload Trap (Waste)

📱 Device: High-end Smartphone

- Viewport Width: 400px
- Device Pixel Ratio: 3.0

🏠 CSS Grid Layout:

- grid-template-columns: repeat(3, 1fr);



Browser Logic (Preloader):

1. Sees 3x device screen → Downloads massive 3x image file.
2. CSS later squishes this 1800px image into a tiny 120px slot.

Pixels Downloaded = $1800^2 = 3,240,000$ px
Pixels Required = $(120 \times 3)^2 = 129,600$ px
Result: 2500% more bandwidth wasted than mathematically necessary!

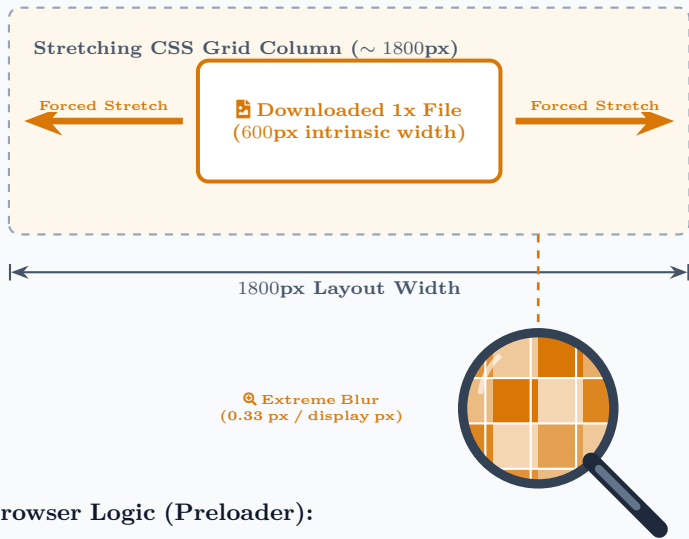
Scenario B: Low DPR, Stretching Grid Column The Extreme Pixelation Trap (Blur)

🖥 Device: Standard Desktop Monitor

- Viewport Width: 1920px
- Device Pixel Ratio: 1.0

✂ CSS Grid Layout:

- grid-template-columns: 1fr; (Wildly stretching)



Browser Logic (Preloader):

1. Sees 1x device screen → Downloads small 1x image file.
2. CSS later forcibly stretches this 600px image to fill 1800px.

$$\text{Information Density} = \frac{\text{Downloaded Width}}{\text{Rendered Layout Width}}$$

$$\text{Density} = \frac{600\text{px}}{1800\text{px}} = 0.33 \text{ pixels per display pixel}$$

Result: Image is mathematically blown up 3x, resulting in severe blur!